Cyprus University of Technology

CSE 226
Algorithms and Complexity

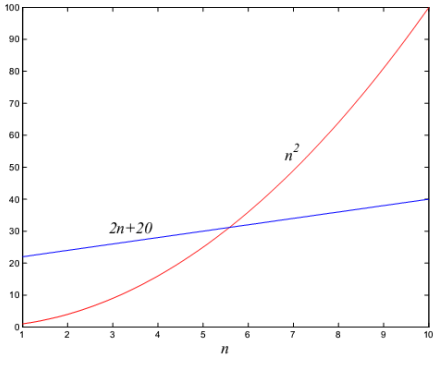
Lecture 2
Asymptotic Analysis
Ασυμπτωτική Ανάλυση

Herodotos Herodotou

Contains edited slides from T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, D. Luebke, and M. Christodoulakis

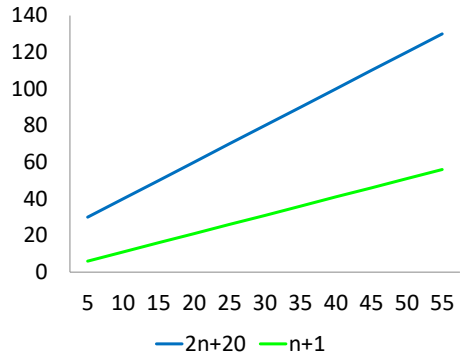
Complexity functions

- Previous lecture: we discussed how to express the running time of an algorithm as a function of input size
- Consider again two algorithms, A and B, where A requires n^2 steps and B $2n + 20$
  - Which one is better?
  - Look at the plots
  - For $n \leq 5$, A is better
  - But... For any value $n > 5$ clearly B is the winner!
  - We say that B **scales** (**κλιμακώνεται**) better



Complexity functions

- Consider now a third algorithm, C, that takes time $n + 1$
- How does C compare to B (with time $2n + 20$)?
  - Certainly C is better
  - But, only by a constant factor!
  - The discrepancy between B and C is tiny compared to that between A and B!



3

Computing the Exact Running Time – Is it worth it?

- Computing the exact running time (with all the constants and lower order terms) is very time consuming
  - See for example the computation of the running time of Insertion-Sort in the previous lecture!

```
   InsertionSort(A, n) {
1:   for i = 2 to n {
2:      key = A[i]
3:      j = i - 1;
4:      while (j > 0) and (A[j] > key) {
5:         A[j+1] = A[j]
6:         j = j - 1
        }
7:      A[j+1] = key
    }
   }
```

The running time is:

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4 \sum_{j=2}^{n} t_j + c_5 \sum_{j=2}^{n} (t_j - 1) + c_6 \sum_{j=2}^{n} (t_j - 1) + c_7(n-1)$$

4

Computing the Exact Running Time – Is it worth it?

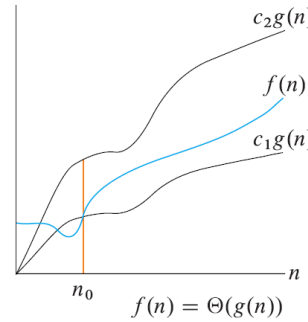
- Computing the exact running time (with all the constants and lower order terms) is very time consuming
  - See for example the computation of the running time of Insertion-Sort in the previous lecture!
- Most often we are not interested in all this detail
- Instead we perform **asymptotic analysis** (**ασυμπτωτική ανάλυση**)
  - Ignore machine-dependent constants
  - Compute only the **growth** (**ρυθμό αύξησης**) of $T(n)$ as $n \to \infty$

Θ-notation

- **Definition:** For a given function $g(n)$ we denote by $\Theta(g(n))$ the **set of functions** (**σύνολο συναρτήσεων**) such that:

  $\Theta(g(n)) = \{ f(n)$: there exist positive constants $c_1, c_2,$ and n_0 such that $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0 \}$

- Notice that since $\Theta(g(n))$ is a set, we could write $f(n) \in \Theta(g(n))$ for a function $f(n)$ that is a member of $\Theta(g(n))$
- Instead, we simply write $f(n) = \Theta(g(n))$ (abuse of notation)



Θ-notation - Example

- An informal way to find the Θ of a function is to throw away lower-order terms and ignore the coefficient of the highest-order term
- Example: $\frac{1}{2}n^2 - 3n = \Theta(n^2)$
  - Formally, we should find positive constants c_1, c_2 and n_0 s.t. $c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2$ for all $n \geq n_0$
  - Divide by n^2 and notice that the right-hand inequality holds for $n \geq 1$ by choosing $c_2 \geq 1/2$, and the left-hand inequality holds for $n \geq 7$ by choosing $c_1 \leq 1/14$
  - So, we choose $c_1 = 1/14$, $c_2 = 1/2$ and $n_0 \geq 7$

Θ-notation

- In general, for any polynomial $p(n) = \sum_{i=0}^{d} a_i n^i$ where a_i are constants and $a_d > 0$, we have $p(n) = \Theta(n^d)$
- For any constant function $f(n) = c$ we have $f(n) = \Theta(1)$ since it is a zero-degree polynomial
- Exercises
  - Prove that $5n + \sqrt{n} = \Theta(n)$
  - Prove that $6n^3 \neq \Theta(n^2)$

Exercises

Prove that $5n + \sqrt{n} = \Theta(n)$

- Need to show that $0 \leq c_1 n \leq 5n + \sqrt{n} \leq c_2 n$ for some $c_1 > 0$, $c_2 > 0$, and for all $n \geq n_o$
- Let $c_1 = 5$. Then $5n \leq 5n + \sqrt{n}$ => $0 \leq \sqrt{n}$, which is true for all $n > 0$
- Let $c_2 = 6$. Then $5n + \sqrt{n} \leq 6n$ => $\sqrt{n} \leq n$, which is true for all $n > 1$
- Hence, true for $c_1 = 5$, $c_2 = 6$, and $n_o = 1$

Exercises

Prove that $6n^3 \neq \Theta(n^2)$

- Proof by **contradiction**
- Suppose it is true. That is, suppose $6n^3 = \Theta(n^2)$
- Then, there is a $c_2 > 0$ such that $6n^3 < c_2 n^2$ for all $n \geq n_0$ => $n < \frac{c_2}{6}$ for all $n \geq n_0$
- This is impossible, hence $6n^3 \neq \Theta(n^2)$

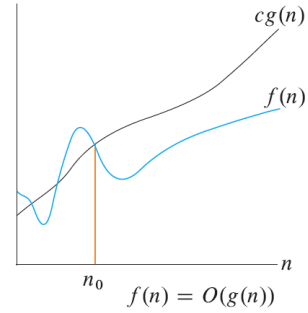
Special names of functions

- A $\Theta(1)$ algorithm is called **constant-time** (**σταθερού χρόνου**) algorithm
- A $\Theta(n)$ is **linear** (**γραμμικός**)
- A $\Theta(n^2)$ is **quadratic** (**τετραγωνικός**)
- A $\Theta(\log n)$ is **logarithmic** (**λογαριθμικός**)
- A $\Theta(c^n)$, where c is a constant, is called **exponential** (**εκθετικός**)
- An algorithm whose complexity is a polynomial is called **polynomial** (**πολυωνυμικός**)

O-notation

- The O-notation provides an **upper bound** (**άνω φράγμα**)
  - while the Θ-notation gives a **tight bound** (**αυστηρό φράγμα**)
- **Definition:**

  $O(g(n)) = \{ f(n)$: there exist positive constants c and n_0 such that $0 \le f(n) \le cg(n)$ for all $n \ge n_0 \}$
- Note that $f(n) = \Theta(g(n))$ **implies that** $f(n) = O(g(n))$
- Why? Because $\Theta(g(n)) \subseteq O(g(n))$
- Examples
  - $3n^3 + 8n^2 + 5n + 12 = O(n^3)$
  - But also, $25n^2 = O(n^3)$ (*why?*)



O-notation

- Is used for bounding the worst-case running time of algorithms
- For example, consider Insertion-Sort
  - The running time is $O(n^2)$ in the worst-case, but also for any other input
  - On the other hand, the running time is $\Theta(n^2)$ in the worst case, but this is not true for every input (e.g., we saw that in the best case it is $\Theta(n)$)

Exercises

- Which of the following is true? Explain
  - $12000n^3 + 45n^{12} = O(n^3)$
  - $9n^2 + n\log n = O(n^2)$
  - $n\log n = O(n^2)$
  - $n\log n = O(n)$
  - $n\log n + 5n = O(n\log n)$
  - $2^{n+1} = O(2^n)$
  - $2^{2n} = O(2^n)$
- Attempt to prove some of the above by using the definitions

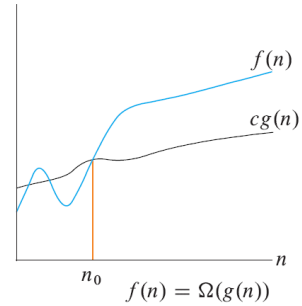
Ω-notation

- As the O-notation provides an upper bound, the Ω-notation provides an asymptotic **lower bound** (**κάτω φράγμα**)
- **Definition:**

  $\Omega(g(n)) = \{ f(n)$: there exist positive constants c and n_0 such that $0 \leq cg(n) \leq f(n)$ for all $n \geq n_0 \}$
- $f(n) = \Theta(g(n))$ **if and only if**

  $f(n) = O(g(n))$ **and**

  $f(n) = \Omega(g(n))$
- Examples
  - $4n^2 + 8 = \Omega(n^2)$
  - $n^2 + 5n + 2 = \Omega(n)$
  - $n \log n = \Omega(n)$



Ω-notation

- The Ω-notation is used to bound (from below) the best-case running time of algorithms
- For example, the best-case running time of Insertion-Sort is $\Omega(n)$, which implies that the running time of Insertion-Sort is $\Omega(n)$ (for any input)
- Exercises
  1. Is it true that Insertion-Sort is $\Omega(n^2)$?
  2. Is it true that the worst-case running time of Insertion-Sort is $\Omega(n^2)$?

o-notation and ω-notation

- The o- and ω-notations are used to denote upper- and lower-bounds respectively, where the bound is **not asymptotically tight** (**δεν είναι ασυμπτωτικά αυστηρό**)
- **Definitions:**

  $\mathrm{o}(\mathrm{g}(n)) = \{ f(n)$: for any positive constant $c > 0$, there exists a constant $n_0 > 0$ such that $0 \leq f(n) < c\mathrm{g}(n)$ for all $n \geq n_0 \}$

  $\omega(\mathrm{g}(n)) = \{ f(n)$: for any positive constant $c > 0$, there exists a constant $n_0 > 0$ such that $0 \leq c\mathrm{g}(n) < f(n)$ for all $n \geq n_0 \}$
- Notice "for any positive constant c"
- Examples
  - $n = \mathrm{o}(n^2)$
  - $n \log n = \omega(n)$

Useful Properties

- **Transitivity** (**Μεταβατικότητα**):
  - $f(n) = \Theta(\mathrm{g}(n))$ **and** $\mathrm{g}(n) = \Theta(h(n))$ **imply** $f(n) = \Theta(h(n))$
  - $f(n) = \mathrm{O}(\mathrm{g}(n))$ **and** $\mathrm{g}(n) = \mathrm{O}(h(n))$ **imply** $f(n) = \mathrm{O}(h(n))$
  - $f(n) = \Omega(\mathrm{g}(n))$ **and** $\mathrm{g}(n) = \Omega(h(n))$ **imply** $f(n) = \Omega(h(n))$
  - $f(n) = \mathrm{o}(\mathrm{g}(n))$ **and** $\mathrm{g}(n) = \mathrm{o}(h(n))$ **imply** $f(n) = \mathrm{o}(h(n))$
  - $f(n) = \omega(\mathrm{g}(n))$ **and** $\mathrm{g}(n) = \omega(h(n))$ **imply** $f(n) = \omega(h(n))$
- **Symmetry** (**Συμμετρία**):
  - $f(n) = \Theta(\mathrm{g}(n))$ **if and only if** $\mathrm{g}(n) = \Theta(f(n))$
- **Transpose symmetry** (**Αναστροφική Συμμετρία**):
  - $f(n) = \mathrm{O}(\mathrm{g}(n))$ **if and only if** $\mathrm{g}(n) = \Omega(f(n))$
  - $f(n) = \mathrm{o}(\mathrm{g}(n))$ **if and only if** $\mathrm{g}(n) = \omega(f(n))$

Exercises

- Which of the following is true?
  - A $\Theta(n)$ algorithm is $O(n)$
  - A $\Theta(n)$ algorithm is $O(n^2)$
  - A $\Theta(n^2)$ algorithm is $O(n^3)$
  - A $\Theta(n)$ algorithm is $O(1)$
  - A $O(1)$ algorithm is $\Theta(1)$
  - A $O(n)$ algorithm is $\Theta(1)$

Exercises

- Determine which of the following bounds are tight bounds (big-O) and which are not tight bounds (small-o). Check to see if any bounds may be wrong.
  - A $\Theta(n)$ algorithm for which we found a $O(n)$ upper bound.
  - A $\Theta(n^2)$ algorithm for which we found a $O(n^3)$ upper bound.
  - A $\Theta(1)$ algorithm for which we found an $O(n)$ upper bound.
  - A $\Theta(n)$ algorithm for which we found an $O(1)$ upper bound.
  - A $\Theta(n)$ algorithm for which we found an $O(2n)$ upper bound.

Exercises

- For the following Θ complexities, write down a tight and a non-tight O bound, and a tight and non-tight Ω bound of your choice, providing they exist.

	O	**Ω**	**o**	**ω**
$\Theta(1)$	$O(1)$	$\Omega(1)$	$o(n)$	--
$\Theta(\sqrt{n})$	$O(\sqrt{n})$	$\Omega(\sqrt{n})$	$o(n)$	$\omega(1)$
$\Theta(n)$	$O(n)$	$\Omega(n)$	$o(n^2)$	$\omega(1)$
$\Theta(n^2)$	$O(n^2)$	$\Omega(n^2)$	$o(n^2 \lg n)$	$\omega(n)$
$\Theta(n^3)$	$O(n^3)$	$\Omega(n^3)$	$o(n^5)$	$\omega(n)$